Doc 3 — Roadmap por sprints 1 a 4 (TP #2 · «Jugátelo»)

Adaptado del enunciado base `3_Roadmap_por_sprints.pdf` (v1.0, 26/10/2026) a la temática de juegos de mesa. Objetivos, historias de usuario, checklists Front/Back, endpoints, datos, criterios de aceptación, tareas a adelantar, estructura de carpetas, riesgos y preguntas. Ver también 1_analisis_funcional.md y 2_modelo_de_datos_mongodb.md.

> **Prioridad para el front:** el roadmap menciona `@Input` / `@Output`, SCSS y la estructura `pages/` + `shared/`. Para el frontend mandan las reglas de código del frontend (input()/output() con default, sin computed, sin comentarios, código en español, etc.). Donde choque con este documento, gana el las reglas de código del frontend. Los marcados con ⚠ abajo son los puntos de choque.

1. Visión general

Cuatro sprints de una semana (fechas [COMPLETAR]).

Sprint	Foco	Endpoints
S1 · Semana 1	Base + registro y login	E-01 · E-02
S2 · Semana 2	Publicaciones, me gusta, Mi Perfil	E-05 · E-06 · E-08 a E-10
S3 · Semana 3	Comentarios + JWT y sesión	E-03 · E-04 · E-07 · E-11 a E-13
S4 · Semana 4	Admin, estadísticas, PWA	E-14 a E-20

1.1 Definición de terminado (todos los sprints)

- [ ] Front y back **deployados y funcionando** en el hosting; las URLs públicas están en el README de cada repo.
- [ ] Variables de entorno configuradas en el hosting (no hay secretos en el repo).
- [ ] Todos los criterios de aceptación del sprint verificados desde la URL pública, no solo en local.
- [ ] Ningún `alert()`, `confirm()` ni `prompt()`: todos los mensajes por ModalService.
- [ ] Status HTTP correctos según Doc 1 cap. 3.3 (nunca 200 en errores ni 400 en éxitos).
- [ ] Diseño consistente con el resto de la app (variables de estilo globales).
- [ ] Commit final etiquetado (p. ej. `sprint-1`) y entregado antes del cierre de la semana.

1.2 Hosting propuesto [COMPLETAR elección]

Capa	Opciones gratuitas	Notas
Front (Angular)	Vercel · Netlify · Firebase Hosting	Fallback de SPA (todas las rutas → index.html) para que `/publicaciones/:id` funcione al recargar. HTTPS (requisito de la PWA).
Back (NestJS)	Render · Railway · Koyeb	Usar el puerto de `process.env.PORT`. En planes gratuitos el servicio puede «dormirse»: la primera petición tarda.
Base de datos	MongoDB Atlas (cluster gratuito)	Ver Doc 2 cap. 8.1.
Imágenes	Cloudinary (plan gratuito)	Carpetas `jugatelo/perfiles` y `jugatelo/publicaciones`.

Verificar las condiciones vigentes de los planes gratuitos al elegir.

2. Tareas que conviene adelantar

Tarea	Exigido en	Hacerlo en	Por qué
Generar el JWT en login y registro (payload `{ id, correo, nombreUsuario, perfil }`, 15 min)	S3	**S1**	En S2 el back necesita saber quién crea, borra o da me gusta; sin token habría que mandar el id en el body y reescribir todo.
JwtAuthGuard + decorador `@UsuarioActual()` en Nest	S3	S2	Lo usan E-05, E-08, E-09 y E-10 para tomar el usuario del token.
authInterceptor (agrega `Authorization: Bearer`)	S3	S2	Todas las llamadas de S2 ya van con token.
TokenService (localStorage) y AuthService con usuario actual	S3	**S1**	Mi Perfil (S2) muestra datos del usuario logueado.
Campo `usuarios.habilitado` (default true)	S4	**S1**	Evita migrar documentos; el guard ya puede chequearlo.
AdminGuard en Nest (perfil «administrador»)	S4	S2	E-08 ya distingue autor o admin; en S4 se reutiliza.
CloudinaryService reutilizable (subir/eliminar)	S1/S2	**S1**	Una sola implementación para perfil (S1) y publicaciones (S2), con carpeta por parámetro.
ModalService + ModalComponent comunes	S1–S4	**S1**	Prohibido `alert()` desde el primer sprint.
FormularioUsuarioComponent reutilizable	S4	**S1**	El alta de usuarios del admin usa «los mismos datos del registro»; solo agrega el radio de perfil.
errorInterceptor (401 → login; 403 deshabilitado)	S3/S4	S2	Centraliza el manejo de errores HTTP.
Guards de Angular (authGuard, adminGuard, invitadoGuard)	S3/S4	S2 (desactivados)	S1 pide navegación «sin límites de accesibilidad» (S-07): se escriben antes y se activan en las rutas en S3/S4.
Esquema Comentario	S3	S2	Mi Perfil (S2) muestra comentarios (S-09).
Pipes y directivas propias	S4	S2–S3	tiempoRelativo, truncar y appImagenRespaldo se usan apenas existen las tarjetas; evita reemplazar código en S4.

3. Sprint 1 — Semana 1

Objetivo: proyectos creados y deployados; registro y login funcionando contra el back propio con MongoDB Atlas y Cloudinary; pantallas base con navegación libre y favicon propio. **Depende de:** nada.

3.1 Historias de usuario

ID	Historia	RF
HU-01	Como visitante quiero registrarme con mis datos y una foto para tener una cuenta en Jugátelo.	RF-01 a RF-05
HU-02	Como visitante quiero iniciar sesión con mi correo y contraseña.	RF-06
HU-03	Como usuario quiero moverme entre Registro, Login, Publicaciones y Mi Perfil.	RF-14
HU-04	Como equipo queremos la app publicada con un favicon propio.	RF-15, RNF-03

3.2 Checklist Front (Angular)

- [ ] `ng new` con componentes standalone y routing; definir paleta, tipografía y espaciados en variables globales.
- [ ] `environment.ts` / `environment.prod.ts` con `apiUrl`.
- [ ] Rutas `/login`, `/registro`, `/publicaciones`, `/mi-perfil`, `/` y `**` (Doc 1 cap. 6). Sin guards (S-07).
- [ ] NavbarComponent con enlaces a todas las pantallas.
- [ ] ModalComponent + ModalService (info, error, confirmación) — *adelantado*.
- [ ] Validadores propios: `contrasenaSegura` (RN-01) y `contrasenasIguales`.
- [ ] FormularioUsuarioComponent reactivo y reutilizable (input `modoAdmin` para S4) con vista previa de imagen — *adelantado*.
- [ ] RegistroComponent: arma FormData (incluye imagenPerfil) y llama a E-01; muestra 409 en modal.
- [ ] LoginComponent: correo + contraseña con validaciones y mensajes; llama a E-02; 401 en modal.
- [ ] AuthService (registro, login, usuarioActual) y TokenService (guardar token) — *adelantado*.
- [ ] PublicacionesComponent y MiPerfilComponent como maquetas con el diseño final.
- [ ] Favicon propio (`src/favicon.ico`) y título de la app.
- [ ] Deploy en el hosting elegido con fallback de SPA.

3.3 Checklist Back (NestJS)

- [ ] `nest new`; ConfigModule con `.env`: `MONGODB_URI`, `JWT_SECRET`, `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`, `FRONT_URL`.
- [ ] `MongooseModule.forRootAsync` contra Atlas.
- [ ] ValidationPipe global `{ whitelist, forbidNonWhitelisted, transform }` y CORS con `FRONT_URL`.
- [ ] Módulos AuthModule, UsuariosModule y PublicacionesModule (este último vacío por ahora).
- [ ] CloudinaryModule + `CloudinaryService.subirImagen(buffer, carpeta)` y `eliminarImagen(publicId)`.
- [ ] Esquema Usuario con índices únicos (Doc 2 cap. 4), incluido `habilitado`.
- [ ] RegistroDto y LoginDto con class-validator (RN-01, S-07, S-24).
- [ ] `POST /auth/registro`: `FileInterceptor('imagenPerfil', memoryStorage)` + `ParseFilePipe` (S-23); verificar unicidad (409); `bcrypt.hash(contrasena, 10)`; subir imagen; guardar; responder 201 `{ usuario, token }`.
- [ ] `POST /auth/login` con `@HttpCode(200)`: `findOne` por correo con `select('+contrasena')`; `bcrypt.compare`; 401 si no coincide; responder `{ usuario, token }`.
- [ ] JwtModule con `expiresIn: '15m'` y payload `{ id, correo, nombreUsuario, perfil }` — *adelantado*.
- [ ] Mapeo de respuesta de usuario sin `contrasena` ni `imagenPerfilPublicId` (RN-09).
- [ ] Traducir error E11000 a 409 (filtro o try/catch en el servicio).
- [ ] Deploy del back; probar E-01 y E-02 con Postman/Thunder Client contra la URL pública.

3.4 Endpoints nuevos

ID	Método	Ruta	Guard	Éxito
E-01	POST	/auth/registro	Público	201 `{ usuario, token }`
E-02	POST	/auth/login	Público	200 `{ usuario, token }`

3.5 Colecciones y campos nuevos

- `usuarios`: todos los campos (nombre, apellido, correo, nombreUsuario, contrasena, fechaNacimiento, descripcion, imagenPerfilUrl, imagenPerfilPublicId, perfil, habilitado, createdAt, updatedAt).

3.6 Criterios de aceptación

- [ ] Registro válido → 201; en Atlas el documento tiene contrasena con prefijo `$2b$` e `imagenPerfilUrl` de Cloudinary.
- [ ] Registro con correo o nombreUsuario existente → 409 y modal con el mensaje correspondiente.
- [ ] Contraseña «abcdefgh» o «abc12345» → error en el formulario y 400 si se envía directo al back.
- [ ] Enviar `perfil = «administrador»` en el registro público → 400 (RN-03).
- [ ] Login correcto → 200 con datos del usuario sin contrasena; contraseña incorrecta → 401 y modal.
- [ ] Se navega entre las cuatro pantallas sin recargar la página.
- [ ] La pestaña del navegador muestra el favicon propio en la URL pública.
- [ ] Búsqueda de «alert(» en el proyecto front sin resultados.

4. Sprint 2 — Semana 2

Objetivo: publicar juegos de mesa y juntadas (con imagen opcional), listarlos paginados con orden por fecha o me gusta y filtros por tipo y zona, reaccionar con me gusta, dar de baja las propias y ver Mi Perfil con datos, foto y últimas 3 publicaciones. **Depende de:** S1 (usuarios, CloudinaryService, JWT generado en login/registro).

4.1 Historias de usuario

ID	Historia	RF
HU-05	Como usuario quiero publicar un juego de mesa que tengo, o una juntada en mi zona, con título, descripción y una imagen opcional.	RF-16
HU-06	Como usuario quiero ver las publicaciones paginadas, por fecha o por cantidad de me gusta, y filtrarlas por tipo y por zona para encontrar juntadas cerca.	RF-17, RF-18, RF-19b, RF-20
HU-07	Como usuario quiero dar y quitar «Me gusta».	RF-21, RF-22
HU-08	Como usuario quiero eliminar mis propias publicaciones.	RF-23
HU-09	Como usuario quiero ver mi perfil con mis datos, mi foto y mis últimas 3 publicaciones con sus comentarios.	RF-29, RF-30

4.2 Checklist Front (Angular)

- [ ] Modelos Publicacion (con `tipo` y campos de juego/juntada), Comentario y `Paginado<T>` en `core/models`.
- [ ] PublicacionesService: crear, `listar(orden, offset, limit, usuarioId?, comentarios?, tipo?, zona?)`, eliminar, darMeGusta, quitarMeGusta.
- [ ] authInterceptor que agrega `Authorization: Bearer <token>` — *adelantado*.
- [ ] PublicacionesComponent: selector de orden («Más recientes» / «Más me gusta»), selector de tipo («Todos» / «Juegos» / «Juntadas»), campo de zona, PaginadorComponent (10 por página, S-10), estado vacío.
- [ ] PublicacionCardComponent con insignia de tipo y datos propios de cada tipo, «Finalizada» en juntadas pasadas (S-29) (⚠ en el roadmap `@Input publicacion` / `@Output eliminada`; en el front usar `input()` / `output()` con default según las reglas de código del frontend) — cada publicación es un componente (RF-20).
- [ ] NuevaPublicacionComponent (en modal o panel): selector de tipo («Tengo un juego» / «Armo una juntada») que muestra los campos del tipo elegido (juego: jugadores mín./máx.; juntada: zona, fecha, cupo), título, descripción, imagen con vista previa; FormData a E-05; validaciones de RN-14.
- [ ] Botón me gusta que alterna E-09 / E-10 según `meGustaDado` y actualiza `cantidadMeGusta` con la respuesta.

- [ ] «Eliminar» visible solo si `usuario._id` coincide con el usuario logueado; confirmación con ModalService.
- [ ] MiPerfilComponent: datos del usuario actual + E-06 con `usuarioId`, `limit = 3`, `comentarios = 3` (S-12).
- [ ] Primeras versiones de los pipes `tiempoRelativo` y `truncar` y de la directiva `appImagenRespaldo`.
- [ ] (Tareas adelantadas de la sección 2: errorInterceptor, guards desactivados.)

4.3 Checklist Back (NestJS)

- [ ] Esquema Publicacion con índices (Doc 2 cap. 5).
- [ ] Esquema Comentario (S-09) registrado en PublicacionesModule.
- [ ] JwtAuthGuard (verifica token, carga el usuario, chequea habilitado) y decorador `@UsuarioActual()` — *adelantado*.
- [ ] ParseObjectIdPipe para `:id` (400 si es inválido).
- [ ] CrearPublicacionDto (tipo + campos condicionales con `@ValidateIf` según RN-14: zona y fechaJuntada obligatorias solo en `juntada`; jugadoresMin/Max solo en `juego`) y ListarPublicacionesQueryDto (orden, tipo, zona, usuarioId, offset, limit, comentarios; `@Type(() => Number)`, `@Min`, `@Max`).
- [ ] `POST /publicaciones`: `FileInterceptor('imagen')`, subir a `jugatelo/publicaciones`, usuario = id del token → 201.
- [ ] `GET /publicaciones` con el pipeline del Doc 2, 7.1 y 7.3 (meGustaDado, autor, comentarios opcionales, filtros tipo y zona).
- [ ] `DELETE /publicaciones/:id`: autor o administrador (ya preparado para S4), `activa = false` → 200.
- [ ] `POST` y `DELETE /publicaciones/:id/me-gusta` con updates atómicos (Doc 2, 7.4): 201 / 409 / 200 / 404.
- [ ] AdminGuard escrito (se usa en S4) — *adelantado*.

4.4 Endpoints nuevos

ID	Método	Ruta	Guard	Éxito
E-05	POST	/publicaciones	JWT	201 publicación creada
E-06	GET	/publicaciones	JWT	200 `{ datos, total, offset, limit }`
E-08	DELETE	/publicaciones/:id	JWT (autor o admin)	200 `{ mensaje }`
E-09	POST	/publicaciones/:id/me-gusta	JWT	201 `{ cantidadMeGusta, meGustaDado: true }`
E-10	DELETE	/publicaciones/:id/me-gusta	JWT	200 `{ cantidadMeGusta, meGustaDado: false }`

4.5 Colecciones y campos nuevos

- `publicaciones`: tipo, titulo, descripcion, jugadoresMin, jugadoresMax, zona, fechaJuntada, cupo, imagenUrl, imagenPublicId, usuario, meGusta, cantidadMeGusta, activa, createdAt, updatedAt + 6 índices.
- `comentarios`: esquema creado (publicacion, usuario, mensaje, createdAt, updatedAt) sin endpoints.

4.6 Criterios de aceptación

- [ ] Con 25 publicaciones, la página 1 muestra 10, la 3 muestra 5 y el paginador indica «Página X de 3».
- [ ] `orden=meGusta` ordena por `cantidadMeGusta` desc; empate por fecha.
- [ ] `GET /publicaciones?usuarioId=<id>` devuelve solo publicaciones de ese usuario.
- [ ] `GET /publicaciones?tipo=juntada&zona=crespo` devuelve solo juntadas cuya zona contiene «crespo» (sin distinguir mayúsculas); `tipo=otro` → 400.
- [ ] Crear una `juntada` sin zona o con fecha pasada → 400; crear un `juego` con zona → 400 (RN-14).
- [ ] Dar me gusta dos veces seguidas (Postman) → 201 y luego 409; `cantidadMeGusta` sube solo 1.

- [ ] Quitar un me gusta no dado → 404 (S-13).
- [ ] DELETE de una publicación ajena con perfil usuario → 403; la propia → 200, desaparece del listado y en Atlas queda `activa: false`.
- [ ] Publicar con imagen de 3 MB o `.gif` → 400 y modal.
- [ ] Mi Perfil muestra todos los datos del usuario, su foto y sus últimas 3 publicaciones (sección comentarios vacía hasta S3).

5. Sprint 3 — Semana 3

Objetivo: pantalla Publicación con comentarios (agregar, editar, cargar más) y JWT completo: token en localStorage, pantalla Cargando con `/auth/autorizar`, aviso de sesión con refresco y redirección ante 401.
Depende de: S2 (publicaciones, JwtAuthGuard, authInterceptor, esquema Comentario).

5.1 Historias de usuario

ID	Historia	RF
HU-10	Como usuario quiero ver un juego o una juntada en grande con sus comentarios.	RF-24, RF-27
HU-11	Como usuario quiero comentar una publicación (por ejemplo, sumarme a una juntada).	RF-26
HU-12	Como usuario quiero editar mis comentarios y que se note que fueron editados.	RF-28
HU-13	Como usuario quiero que la app recuerde mi sesión al volver a abrirla.	RF-08, RF-09
HU-14	Como usuario quiero que me avisen antes de que venza la sesión y poder extenderla.	RF-10
HU-15	Como usuario quiero volver al login si mi sesión ya no es válida.	RF-11, RF-12

5.2 Checklist Front (Angular)

- [ ] Rutas `/cargando` (inicial) y `/publicaciones/:id`; activar authGuard e invitadoGuard.
- [ ] CargandoComponent: SpinnerComponent + E-03; redirige a /publicaciones o /login (CU-03).
- [ ] TokenService con localStorage (clave `token`) y decodificación del `exp`.
- [ ] SesionService: al iniciar o extender, programa el aviso en (exp – 5 min) — equivale a 10 min tras el login; modal «¿Querés extender tu sesión?»; Sí → E-04 y reprograma; No → cierre al vencer (S-15).
- [ ] Revisar el tiempo restante al volver a la pestaña (evento `visibilitychange`) por si el timer se demoró.
- [ ] errorInterceptor: 401 → borrar token, detener SesionService, modal y /login (RN-10).
- [ ] PublicacionComponent: E-07 + ComentariosComponent.
- [ ] ComentariosComponent: carga inicial (offset 0, limit 5), «Cargar más» acumulativo con descarte de repetidos por `_id`; se oculta cuando cargados = total.
- [ ] ComentarioItemComponent: «(editado)» si `modificado`; edición en línea solo para el autor (E-13).
- [ ] Formulario de nuevo comentario (1–500) → E-12 e inserción al inicio de la lista.
- [ ] Mi Perfil ya muestra comentarios reales.
- [ ] Para probar: variable de entorno que acorte los tiempos de sesión en desarrollo.

5.3 Checklist Back (NestJS)

- [ ] `POST /auth/autorizar` (`@HttpCode(200)`): JwtAuthGuard → devuelve `{ usuario }`.
- [ ] `POST /auth/refrescar` (`@HttpCode(200)`): firma un token nuevo con la misma payload y 15 min.
- [ ] Confirmar que login y registro devuelven token (ya hecho en S1).
- [ ] `GET /publicaciones/:id` (S-08): pipeline de 7.1 con `$match` por `_id`; 404 si no existe o `activa = false`.
- [ ] ComentariosController en PublicacionesModule: GET / POST / PUT según Doc 2, 7.6.
- [ ] CrearComentarioDto / EditarComentarioDto (mensaje 1–500) y PaginacionQueryDto (offset, limit).
- [ ] PUT: 403 si no es el autor; marca `modificado = true`.

- [ ] Asegurar que JwtAuthGuard responde 401 ante token ausente, mal formado, firma inválida o vencido.

5.4 Endpoints nuevos

ID	Método	Ruta	Guard	Éxito
E-03	POST	/auth/autorizar	JWT	200 `{ usuario }`
E-04	POST	/auth/refrescar	JWT	200 `{ token }`
E-07	GET	/publicaciones/:id	JWT	200 publicación
E-11	GET	/publicaciones/:publicacionId/comentarios	JWT	200 `{ datos, total, offset, limit }`
E-12	POST	/publicaciones/:publicacionId/comentarios	JWT	201 comentario
E-13	PUT	/publicaciones/:publicacionId/comentarios/:comentarioId	JWT (autor del comentario)	200 comentario con `modificado: true`

5.5 Colecciones y campos nuevos

- `comentarios`: empiezan a crearse documentos; campo `modificado`; índice `{ publicacion: 1, createdAt: -1 }`.

5.6 Criterios de aceptación

- [ ] El token decodificado (p. ej. en jwt.io) tiene `{ id, correo, nombreUsuario, perfil }` y `exp - iat = 900`.
- [ ] Recargar la app con token válido → Cargando → /publicaciones; sin token → /login.
- [ ] Token alterado en localStorage → la próxima petición devuelve 401 y la app va al login con aviso.
- [ ] A los 10 minutos del login aparece el modal; «Extender» devuelve un token nuevo y el aviso vuelve a programarse.
- [ ] Los comentarios llegan del más reciente al más antiguo, de a 5; «Cargar más» agrega sin borrar los anteriores.
- [ ] Editar un comentario ajeno por Postman → 403; el propio → 200 y se ve «(editado)».
- [ ] GET de comentarios de una publicación dada de baja → 404.
- [ ] Visitar /mi-perfil sin sesión redirige al login.

6. Sprint 4 — Semana 4

Objetivo: funciones de administrador (baja de cualquier publicación, Dashboard Usuarios, Dashboard Estadísticas), bloqueo de usuarios deshabilitados, PWA, 3 pipes y 3 directivas propias. **Depende de:** S1 (usuarios, FormularioUsuarioComponent), S2 (AdminGuard, baja de publicaciones), S3 (JWT, errorInterceptor, comentarios para las estadísticas).

6.1 Historias de usuario

ID	Historia	RF
HU-16	Como administrador quiero dar de baja cualquier publicación para moderar el contenido.	RF-25
HU-17	Como administrador quiero ver el listado de usuarios.	RF-31, RF-32
HU-18	Como administrador quiero crear usuarios eligiendo si son «usuario» o «administrador».	RF-33

ID	Historia	RF
HU-19	Como administrador quiero deshabilitar y rehabilitar usuarios.	RF-34, RF-35
HU-20	Como usuario deshabilitado quiero saber por qué no puedo ingresar.	RF-13
HU-21	Como administrador quiero ver estadísticas en un lapso que elija.	RF-36 a RF-40
HU-22	Como usuario quiero instalar la app (PWA).	RNF-04

6.2 Checklist Front (Angular)

- [ ] adminGuard en `/dashboard/usuarios` y `/dashboard/estadisticas`; enlace «Dashboard» con `*appSoloAdmin`.
- [ ] Botón «Eliminar» en PublicacionCardComponent y PublicacionComponent también para administradores.
- [ ] DashboardUsuariosComponent: tabla (foto, nombreCompleto, nombreUsuario, correo, perfil, estado), acciones Deshabilitar / Habilitar con confirmación; el propio usuario no puede deshabilitarse.
- [ ] FormularioUsuarioComponent en modo admin: radio buttons perfil «usuario» / «administrador» → E-15.
- [ ] UsuariosService y EstadisticasService.
- [ ] DashboardEstadisticasComponent: filtro desde/hasta (defecto últimos 30 días, S-18), `forkJoin` de E-18, E-19 y E-20, tres gráficos.
- [ ] LoginComponent y errorInterceptor: 403 de usuario deshabilitado → modal con el mensaje de RN-07.
- [ ] `ng add @angular/pwa`: manifest con nombre «Jugátelo», colores e íconos; en `ngsw-config` no cachear `/auth/*`.
- [ ] Pipes finales: tiempoRelativo, truncar, nombreCompleto. Directivas finales: appSoloAdmin, appImagenRespaldo, appMostrarContrasena.
- [ ] Repaso visual completo de uniformidad (RNF-01).

6.3 Checklist Back (NestJS)

- [ ] UsuariosController con `@UseGuards(JwtAuthGuard, AdminGuard)`: GET, POST, DELETE `/:id`, POST `/:id/habilitar` (`@HttpCode(200)`).
- [ ] CrearUsuarioDto = campos de RegistroDto + `perfil` (`@IsIn(['usuario', 'administrador'])`).
- [ ] Reutilizar la lógica de alta del registro (hash, unicidad, Cloudinary) desde UsuariosService.
- [ ] `DELETE /usuarios/:id`: `habilitado = false`; 400 si es el propio administrador (S-17).
- [ ] Login: si `habilitado = false` → ForbiddenException con el mensaje de RN-07 (403).
- [ ] EstadisticasController en PublicacionesModule con `@UseGuards(JwtAuthGuard, AdminGuard)` y RangoFechasQueryDto (desde ≤ hasta).
- [ ] Aggregations de E-18, E-19 y E-20 (Doc 2, 7.7).
- [ ] Verificar que `DELETE /publicaciones/:id` acepta administradores.

6.4 Endpoints nuevos

ID	Método	Ruta	Guard	Éxito
E-14	GET	/usuarios	Admin	200 `[ usuario ]`
E-15	POST	/usuarios	Admin	201 usuario
E-16	DELETE	/usuarios/:id	Admin	200 usuario con `habilitado: false`
E-17	POST	/usuarios/:id/habilitar	Admin	200 usuario con `habilitado: true`
E-18	GET	/estadisticas/publicaciones-por-usuario	Admin	200 `[ { usuarioId, nombreUsuario, cantidad } ]`

ID	Método	Ruta	Guard	Éxito
E-19	GET	/estadisticas/comentarios	Admin	200 `{ total, porDia: [ { fecha, cantidad } ] }`
E-20	GET	/estadisticas/comentarios-por-publicacion	Admin	200 `[ { publicacionId, titulo, cantidad } ]`

6.5 Colecciones y campos nuevos

- `usuarios`: se empieza a usar `habilitado` y perfil «administrador» (campos creados en S1).
- `publicaciones` y `comentarios`: índice `{ createdAt: 1 }` para los rangos de fechas.

6.6 Criterios de aceptación

- [ ] Con perfil usuario, `/dashboard/usuarios` redirige a /publicaciones y `GET /usuarios` por Postman responde 403.
- [ ] El admin ve el botón eliminar en publicaciones ajenas; al usarlo desaparece la publicación y sus comentarios (GET de sus comentarios → 404).
- [ ] Alta de usuario con perfil «administrador» → 201; ese usuario al loguearse ve el Dashboard.
- [ ] Deshabilitar a un usuario → su login responde 403 y muestra el mensaje; al rehabilitarlo puede ingresar.
- [ ] Un usuario deshabilitado con sesión abierta recibe 403 en la siguiente petición y la app lo saca (S-16).
- [ ] Cambiar el lapso modifica los tres gráficos; desde > hasta muestra validación y el back responde 400.
- [ ] Los tres gráficos son de tipos distintos (barras, líneas, dona).
- [ ] Lighthouse / Chrome muestra la app como instalable desde la URL pública (HTTPS).
- [ ] Se pueden señalar en el código 3 pipes y 3 directivas propias en uso.

6.7 Gráficos (Dashboard Estadísticas)

Librería sugerida: **Chart.js** con **ng2-charts** (directiva `baseChart`, compatible con standalone).

Estadística	Endpoint	Tipo	Datos
Publicaciones por usuario en el lapso	E-18	Barras (horizontal si hay muchos usuarios)	labels = nombreUsuario · data = cantidad
Comentarios realizados en el lapso	E-19	Líneas + tarjeta con el total	labels = cada día del rango (días sin datos en 0) · data = cantidad
Comentarios en cada publicación en el lapso	E-20	Dona (o torta)	labels = título (truncar) · data = cantidad

```
npm install chart.js ng2-charts

// app.config.ts
providers: [ provideCharts(withDefaultRegisterables()), ... ]

// dashboard-estadisticas.component.html
<canvas baseChart [type]="'bar'" [data]="publicacionesPorUsuario" [options]="opcionesBarras">
</canvas>
```

7. Estructura de carpetas sugerida

7.1 Front — Angular (standalone)

⚠ Es una **sugerencia**; para el front prevalecen las reglas de código del frontend (carpetas técnicas `core/`, `shared/`, `features/`, `layout/`; código de dominio en español).

```
src/
├─ app/
│  ├─ app.config.ts        # provideRouter, provideHttpClient(withInterceptors([...])), charts,
SW
│  ├─ app.routes.ts        # rutas del Doc 1, cap. 6
│  ├─ core/
│  │  ├─ guards/           # auth.guard.ts · admin.guard.ts · invitado.guard.ts
│  │  ├─ interceptors/     # auth.interceptor.ts · error.interceptor.ts
│  │  ├─ models/           # usuario.model.ts · publicacion.model.ts · comentario.model.ts
│  │  │                    # paginado.model.ts · estadisticas.model.ts
│  │  └─ services/         # auth.service.ts · token.service.ts · sesion.service.ts ·
modal.service.ts
│  │                       # publicaciones.service.ts · comentarios.service.ts
│  │                       # usuarios.service.ts · estadisticas.service.ts
│  ├─ shared/
│  │  ├─ components/       # navbar/ · modal/ · spinner/ · paginador/ · formulario-usuario/
│  │  │                    # publicacion-card/ · nueva-publicacion/ · comentarios/ · comentario-
item/
│  │  ├─ pipes/            # tiempo-relativo.pipe.ts · truncar.pipe.ts · nombre-completo.pipe.ts
│  │  ├─ directives/       # solo-admin.directive.ts · imagen-respaldo.directive.ts
│  │  │                    # mostrar-contrasena.directive.ts
│  │  └─ validators/       # contrasena-segura.validator.ts · contrasenas-iguales.validator.ts
│  └─ pages/
│     ├─ cargando/ · login/ · registro/ · publicaciones/ · publicacion/ · mi-perfil/
│     └─ dashboard/ (usuarios/ · estadisticas/)
├─ environments/           # environment.ts · environment.prod.ts (apiUrl)
├─ styles/                 # _variables.scss · _botones.scss · _formularios.scss
├─ favicon.ico
└─ manifest.webmanifest    # (S4, generado por @angular/pwa)
```

7.2 Back — NestJS

```
src/
├─ main.ts                 # ValidationPipe global, CORS (FRONT_URL), PORT
├─ app.module.ts           # ConfigModule, MongooseModule.forRootAsync, módulos
├─ comun/
│  ├─ cloudinary/          # cloudinary.module.ts · cloudinary.provider.ts ·
cloudinary.service.ts
│  ├─ pipes/               # parse-object-id.pipe.ts
│  ├─ filters/             # mongo-duplicado.filter.ts (E11000 -> 409)
│  └─ validators/          # imagen.validator.ts (tipo y tamaño, S-23)
├─ auth/
│  ├─ auth.module.ts · auth.controller.ts · auth.service.ts   # /auth (E-01 a E-04)
│  ├─ guards/              # jwt-auth.guard.ts · admin.guard.ts
│  ├─ decorators/          # usuario-actual.decorator.ts
│  ├─ interfaces/          # jwt-payload.interface.ts { id, correo, nombreUsuario, perfil }
│  └─ dto/                 # registro.dto.ts · login.dto.ts
├─ usuarios/
│  ├─ usuarios.module.ts · usuarios.controller.ts · usuarios.service.ts   # /usuarios (E-14 a E-
17)
│  ├─ schemas/             # usuario.schema.ts
│  └─ dto/                 # crear-usuario.dto.ts
└─ publicaciones/
   ├─ publicaciones.module.ts
   ├─ controllers/         # publicaciones.controller.ts   /publicaciones (E-05 a E-10)
   │                       # comentarios.controller.ts
/publicaciones/:publicacionId/comentarios
   │                       # estadisticas.controller.ts    /estadisticas (E-18 a E-20)
   ├─ services/            # publicaciones.service.ts · comentarios.service.ts ·
estadisticas.service.ts
   ├─ schemas/             # publicacion.schema.ts · comentario.schema.ts
   └─ dto/                 # crear-publicacion.dto.ts · listar-publicaciones.query.dto.ts
                           # crear-comentario.dto.ts · editar-comentario.dto.ts
                           # paginacion.query.dto.ts · rango-fechas.query.dto.ts
```

Nota: AuthModule importa UsuariosModule (para buscar y crear usuarios) y exporta JwtAuthGuard, AdminGuard y JwtModule; PublicacionesModule importa AuthModule y CloudinaryModule.

7.3 Ejemplo de DTO con class-validator — `auth/dto/registro.dto.ts`

```
export class RegistroDto {
  @IsString() @Length(2, 50) @Matches(/^[\p{L} ]+$/u) nombre: string;
  @IsString() @Length(2, 50) @Matches(/^[\p{L} ]+$/u) apellido: string;
  @IsEmail() @Transform(({ value }) => value?.trim().toLowerCase()) correo: string;
  @IsString() @Length(3, 20) @Matches(/^[a-zA-Z0-9._]+$/) nombreUsuario: string;
  @Matches(/^(?=.*[A-Z])(?=.*\d).{8,}$/, {
    message: 'La contraseña debe tener al menos 8 caracteres, una mayúscula y un número' })
  contrasena: string;
  @IsString() repetirContrasena: string; // igualdad validada en el servicio (S-07)
  @Type(() => Date) @IsDate() @MaxDate(() => new Date()) fechaNacimiento: Date;
  @IsOptional() @IsString() @MaxLength(200) descripcion?: string;
}
```

8. Riesgos

Riesgo	Impacto	Mitigación
El back gratuito «duerme» y la primera petición tarda	Cargando se queda esperando o vence el timeout en la corrección	Mensaje claro en Cargando y botón Reintentar; abrir la URL del back unos minutos antes de corregir.
Mi Perfil (S2) pide comentarios antes del controller de comentarios (S3)	Requisito de S2 incompleto	Esquema Comentario en S2 + parámetro `comentarios` de E-06 (S-09); confirmar con el docente.
«Cargar más» con offset y comentarios nuevos en el medio	Comentarios repetidos en pantalla	Descartar repetidos por `_id` en ComentariosComponent.
Timers del navegador demorados en pestañas inactivas	El aviso de sesión aparece tarde	Programar según el `exp` del token y recalcular en `visibilitychange`.
Token en localStorage expuesto a XSS	Robo de sesión	No usar `innerHTML` con contenido de usuarios; dejar que Angular sanee; vencimiento corto (15 min).
Atlas con acceso 0.0.0.0/0	Base expuesta si se filtra la URI	Usuario de base con permisos mínimos, contraseña fuerte, URI solo en variables de entorno.
Índices únicos no creados en Atlas	Duplicados de correo/nombreUsuario; no aparece el 409	Verificar la pestaña Indexes antes de cada entrega; chequeo previo en el servicio.
Me gusta concurrentes (doble clic)	Contador desfasado	Updates atómicos condicionales (Doc 2, 7.4) y botón deshabilitado durante la petición.
POST de Nest devuelve 201 por defecto	Status incorrecto en login, autorizar, refrescar y habilitar	`@HttpCode(HttpStatus.OK)` en esos métodos.
Diferencia horaria en estadísticas	Comentarios de la noche contados en el día siguiente	Rango con `-03:00` y `timezone` en `$dateToString` (S-18).
Service worker cacheando respuestas de la API	Datos viejos o sesión inconsistente	No incluir `/auth/*` ni la API en `assetGroups`; `dataGroups` con estrategia freshness o sin cache.
Imagen subida a Cloudinary y fallo posterior al guardar	Imágenes huérfanas	Validar todo antes de subir; si falla el guardado, `eliminarImagen(publicId)`.
Límites del plan gratuito de Cloudinary	Rechazo de subidas	Máximo 2 MB por imagen (S-23).

9. Preguntas a confirmar con el docente

1. ¿La edición del perfil propio se evalúa en los Sprints 1–4? ¿Qué campos serían editables? (S-20)

2. En S2, Mi Perfil pide comentarios pero el controller de comentarios es de S3: ¿alcanza con mostrar la sección vacía hasta S3? (S-09)
3. ¿La imagen de perfil es obligatoria en el registro? (S-05)
4. ¿Se acepta una sola imagen por publicación o se espera poder adjuntar varias? (S-03)
5. Temática «Jugátelo»: ¿está bien resolver juegos y juntadas en una sola colección `publicaciones` con un campo `tipo`, o se espera una colección aparte para las juntadas? ¿Alcanza con `zona` como texto libre? (S-27, S-28)
6. ¿Qué status se espera para un me gusta repetido (409) y para quitar uno que no existe (404)? (S-13)
7. Estadísticas: ¿se incluyen publicaciones dadas de baja? ¿«Comentarios en cada publicación en un lapso» son los creados en el lapso agrupados por publicación? (S-14, S-18)
8. Si el usuario no extiende la sesión, ¿debe cerrarse automáticamente al vencer el token? (S-15)
9. ¿Un administrador puede deshabilitar a otro administrador? ¿Y a sí mismo? (S-17)
10. ¿La paginación de publicaciones requiere números de página o alcanza con Anterior/Siguiente? (S-10)
11. ¿Hay cantidad mínima de elementos por página o de comentarios por carga? (S-10, S-11)

10. Anexo — Nomenclatura común

Idéntica a la del Doc 1 (sección 9): ver 1_analisis_funcional.md.